

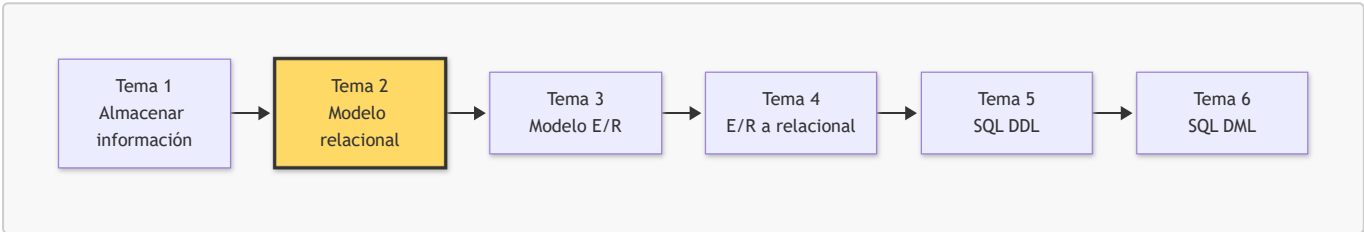
Tema 2. Bases de Datos Relacionales

«Los usuarios de los grandes bancos de datos no deben preocuparse de cómo están organizados los datos por dentro.»
— **Edgar F. Codd**, 1970 (idea central de su artículo, citada de forma resumida)

1º DAW · Bases de Datos (módulo 0484) · IES Ntra. Sra. de los Remedios de Ubrique · Curso 2026/2027

0. Antes de empezar

0.1. ¿Dónde encaja este tema?



Este tema es el **cimiento** de todo lo que viene después. Si lo entiendes bien, crear tablas en SQL será casi mecánico.

0.2. Marco normativo

Norma	Qué dice
Real Decreto 405/2023, de 29 de mayo (BOE 3-6-2023)	Redacta de nuevo el módulo 0484 Bases de datos (105 h, 12 ECTS). Es la normativa vigente.
Real Decreto 686/2010, de 20 de mayo	Título original de Técnico Superior en DAW (redacción anterior).

0.3. Resultados de aprendizaje y criterios de evaluación que trabajamos

RA	Resultado de aprendizaje	Peso en este tema
RA1	Reconoce los elementos de las bases de datos analizando sus funciones y valorando la utilidad de los sistemas gestores.	☆☆
RA2	Crea bases de datos definiendo su estructura y las características de sus elementos según el modelo relacional.	☆☆☆
RA4	Modifica la información almacenada en la base de datos...	☆
RA6	Diseña modelos relacionales normalizados interpretando diagramas entidad/relación.	☆☆☆

Criterios de evaluación concretos:

Criterio	Se trabaja en el apartado
RA1.b) Identificar los tipos de BD según el modelo de datos.	1
RA2.b) Crear las tablas y las relaciones entre ellas.	2, 6, 8
RA2.d) Definir los campos clave en las tablas.	3
RA2.e) Implantar las restricciones reflejadas en el diseño lógico.	5, 6
RA4.h) Adoptar medidas para mantener la integridad y consistencia.	5, 6
RA6.b) Identificar las tablas del diseño lógico.	4
RA6.c) Identificar los campos de las tablas.	2, 4
RA6.d) Analizar las relaciones entre las tablas.	4, 6
RA6.e) Identificar los campos clave.	3
RA6.f) Aplicar reglas de integridad.	5, 6
RA6.h) Analizar y documentar restricciones que no pueden plasmarse en el diseño lógico.	5.5

Contenidos básicos oficiales: modelo de datos · terminología del modelo relacional · claves primarias · restricciones de validación · el valor **NULL** · claves ajenas · integridad referencial · restricciones semánticas del modelo relacional.

0.4. Qué serás capaz de hacer al terminar

- ☐ Explicar por qué el modelo relacional ganó la partida.
- ☐ Distinguir relación, atributo, dominio, tupla, grado y cardinalidad.
- ☐ Reconocer superclaves, claves candidatas, primarias, alternativas y ajenas.
- ☐ Leer y escribir un **grafo relacional**.
- ☐ Decidir qué pasa al borrar o modificar datos relacionados.
- ☐ Traducir todo ello a SQL.

0.5. Nuestro caso de estudio: el instituto

A lo largo del tema usaremos **un único ejemplo**, que conoces bien: **tu propio instituto**.

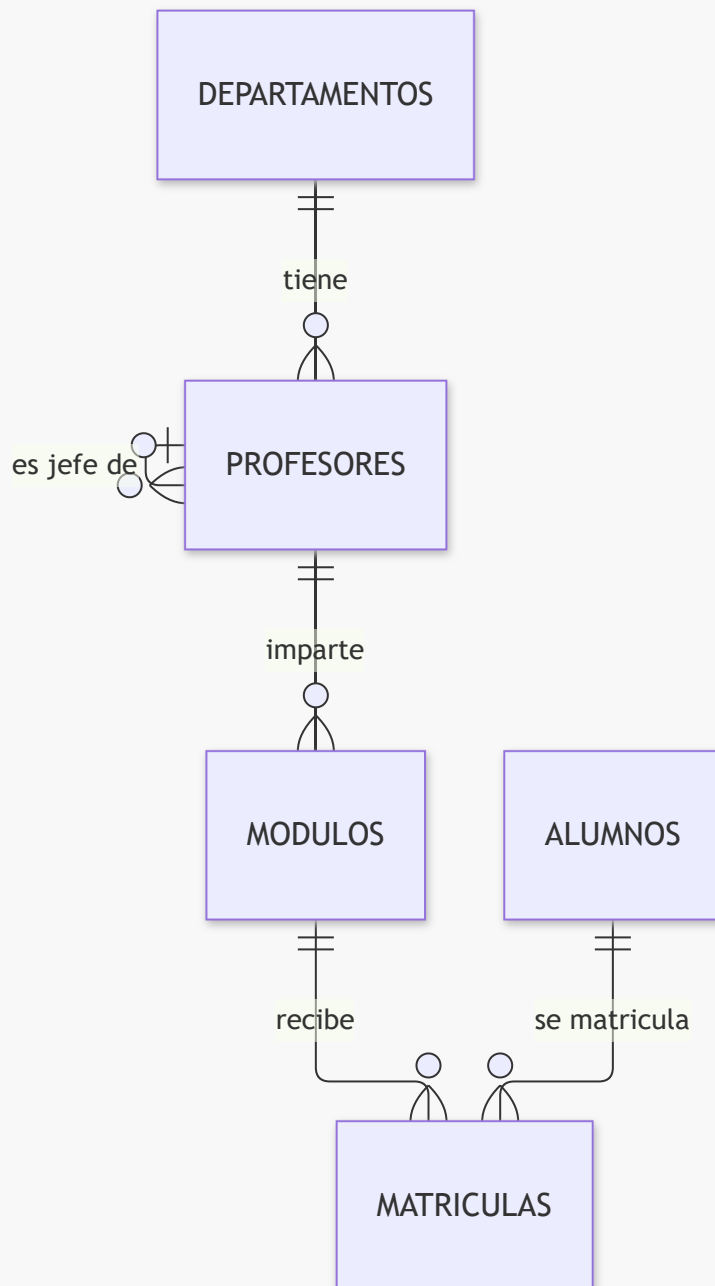


Tabla	Qué guarda
DEPARTAMENTOS	Informática, Inglés, FOL...
PROFESORES	Profesorado (y quién es el jefe de cada uno).
MODULOS	Asignaturas: Bases de Datos, Programación...
ALUMNOS	Alumnado del centro.
MATRICULAS	Qué alumno cursa qué módulo y con qué nota.

1. El modelo relacional: de dónde viene

1.1. Antes de Codd: el caos de los ficheros

Imagina que la secretaria guarda los datos en **fichas de papel dentro de cajones**.

- La ficha de Ana dice: «*Ana, 1º DAW, Ubrique*».
- Si Ana se muda, hay que cambiarlo en **cada** cajón donde aparezca.
- Si el cajón se reorganiza, **todo el mundo** tiene que aprender dónde está cada cosa.

Los primeros modelos de datos (**jerárquico** y **en red**) eran así: para consultar algo había que **conocer el camino físico** hasta el dato.

Modelo	Cómo se accede al dato	Problema
Jerárquico	Recorriendo un árbol padre → hijo.	Rígido; un hijo solo tiene un padre.
En red	Siguiendo punteros entre registros.	Complejo; hay que conocer la estructura física.
Relacional	Pidiendo qué quieres, no dónde está.	Ninguno grave: por eso triunfó.

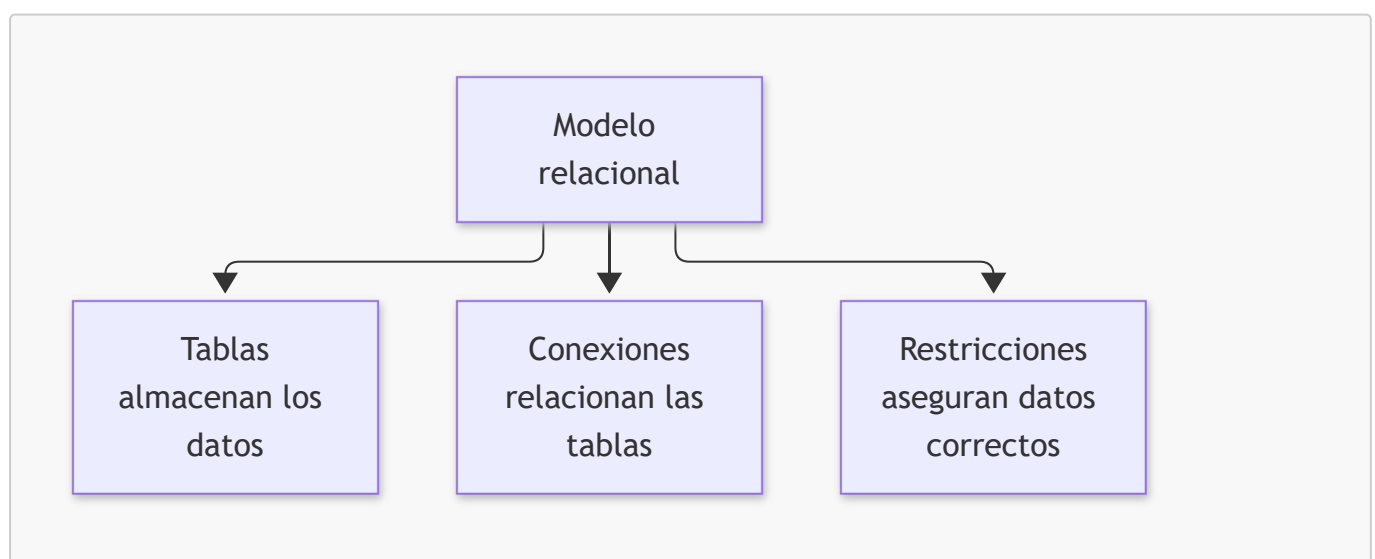
1.2. La idea de Codd (1970)

💡 **Idea clave:** Tú dices **qué** quieres. El sistema decide **cómo** conseguirlo.

Año	Hito
1970	Edgar F. Codd (IBM) publica su artículo en <i>Communications of the ACM</i> .
1970s	IBM no lo apoya al principio; Oracle lo implementa comercialmente.
1974	Nace SQL (entonces SEQUEL) en IBM.
1980s en adelante	El modelo relacional domina el mercado.

Codd era **matemático**, y por eso el modelo usa términos de la **teoría de conjuntos** y la **lógica**: *relación, tupla, dominio...*

1.3. Los tres pilares



1.4. Objetivos del modelo (con ejemplos)

Objetivo	Significado	Ejemplo cotidiano
Independencia física	Cómo se guardan los datos no afecta a cómo se usan.	Cambias el disco duro del servidor y la app sigue igual.
Independencia lógica	Cambiar la estructura no rompe las aplicaciones.	Añades una columna email y la app de notas sigue funcionando.
Flexibilidad	Cada usuario ve lo que necesita (vistas).	Un profesor ve sus notas; el director ve las de todos.
Sencillez	Todo son tablas.	Si sabes leer una hoja de cálculo, sabes leer una BD.
Base teórica	Matemáticas rigurosas.	Se puede <i>demostrar</i> que una consulta es correcta.

1.5. ¿Sigue vigente hoy?

Sí. Casi todos los SGBD usan este modelo:

- **Libres:** MySQL, MariaDB, PostgreSQL.
- **Comerciales:** Oracle, SQL Server, Informix.

Junto a él conviven las bases de datos **no relacionales** (NoSQL: documentos, clave-valor, grafos), que veremos en el Tema 9 (RA7).

Si tus datos...	Elige
Son estructurados y con relaciones claras (notas, facturas, bancos)	Relacional
Son muy variables o masivos (redes sociales, sensores, catálogos flexibles)	NoSQL

2. La estructura: tablas con nombres científicos

2.1. Diccionario de traducción

El modelo usa palabras "serias". Aquí tienes el diccionario:

Término formal	En SQL	En hoja de cálculo	Pista para recordarlo
Relación	Tabla	Hoja	La hoja completa
Atributo	Columna	Cabecera de columna	"Una característica"
Tupla	Fila	Fila	"Un ejemplar concreto"
Dominio	Tipo de dato + restricciones	Formato de celda	"Qué valores son válidos"
Grado	Nº de columnas	Ancho	
Cardinalidad	Nº de filas	Alto	

⚠ **Cuidado con el nombre.** La "relación" de Codd es una **tabla**. La "relación" del modelo E/R (Chen) es una **asociación** entre entidades. ¡No son lo mismo!

2.2. Nuestra primera relación: ALUMNOS

id_alumno	nombre	apellidos	fecha_nac	localidad
1	Aya	Essadik	2007-03-12	Ubrique
2	César	Gallardo	2006-11-02	El Bosque
3	Maitane	Navarro	2007-05-21	Ubrique
4	Hugo	Valderrama	2006-09-30	Villamartín

Identifica cada parte:

ATRIBUTOS (columnas)						
	id_alumno	nombre	apellidos	fecha_nac	localidad	← CABECERA
TUPLA →	1	Aya	Essadik	2007-03-12	Ubrique	← CUERPO
TUPLA →	2	César	Gallardo	2006-11-02	El Bosque	
TUPLA →	3	Maitane	Navarro	2007-05-21	Ubrique	
TUPLA →	4	Hugo	Valderrama	2006-09-30	Villamartín	

2.3. Grado y cardinalidad

Medida	¿Qué cuenta?	En ALUMNOS
Grado	Atributos (columnas)	5
Cardinalidad	Tuplas (filas)	4

- El **grado** casi nunca cambia (solo si alteras la tabla).
- La **cardinalidad** cambia **constantemente**: cada alta o baja la modifica.

🧠 **Truco:** *Grado = Columnas* (la "G" de "Grado" va con la "C" de... ¡no!). Mejor: **Grado = Golpe de vista a lo ancho**; **Cardinalidad = Cantidad de filas**.

Según el grado, una relación es: **unaria** (1), **binaria** (2), **ternaria** (3) o **n-aria** (n).

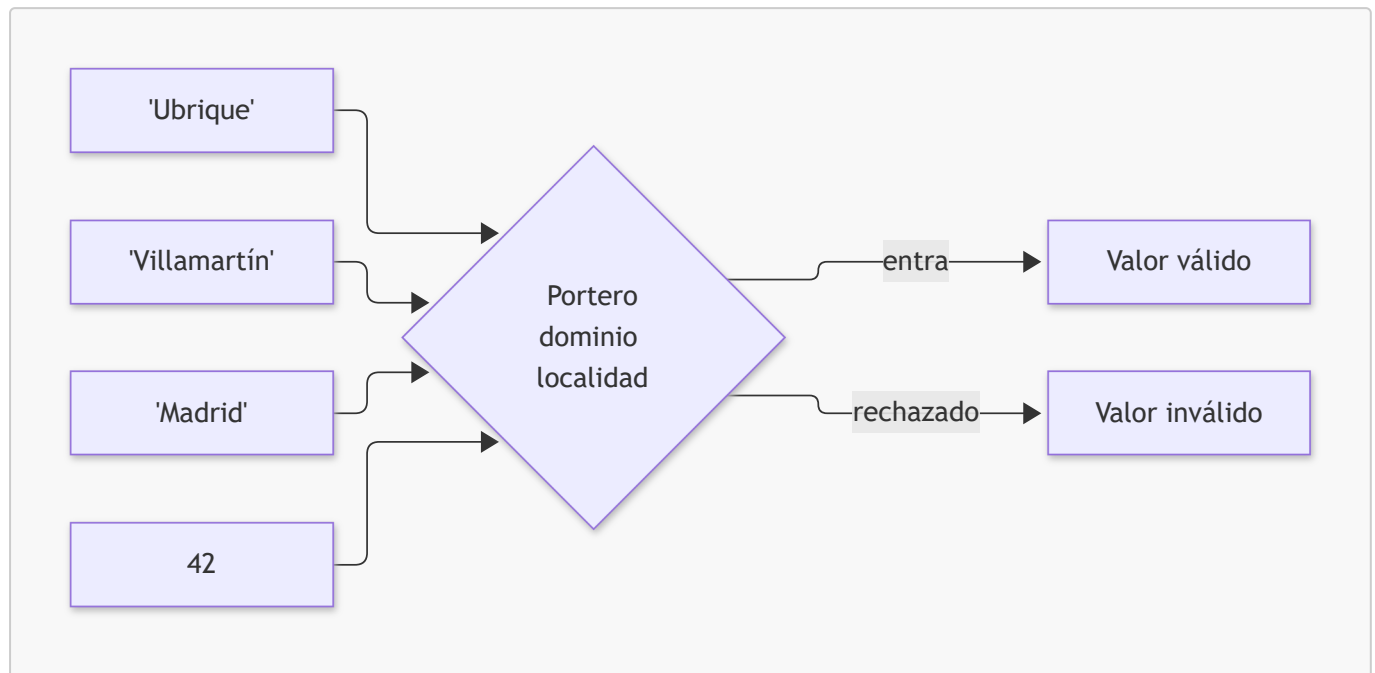
2.4. Atributos

Reglas que cumplen **todos** los atributos:

- ✅ Nombre **único** dentro de la tabla.
- ✅ Pueden estar en **cualquier orden**.
- ✅ Todas las tuplas tienen **los mismos** atributos.
- ✅ Cada valor pertenece al **dominio** del atributo (o es **NULL**).

2.5. Dominios: el "club" de valores permitidos

Un **dominio** es el conjunto de valores que un atributo **puede** tomar. Piensa en una **discoteca con portero**: solo entran los que cumplen las reglas.



Los valores del dominio son:

- **Homogéneos:** todos del mismo tipo.
- **Atómicos:** indivisibles (no se puede guardar "Ubrique, Cádiz" en un solo campo si quieres tratar localidad y provincia por separado).

Tipo de dominio	Descripción	Ejemplo
General	Entre un mínimo y un máximo.	<i>nota:</i> de 0 a 10.
Restringido	Lista cerrada de valores.	<i>turno:</i> 'Mañana' o 'Tarde'.

Dos dominios distintos, ¿se pueden comparar?

-  Comparar *fecha_nac* con *nota* no tiene sentido, aunque ambos sean números/fechas.
-  Multiplicar *precio_hora* × *horas* sí tiene sentido: da el coste total.

En la práctica, los SGBD **no implementan dominios completos**. Los simulan con **tipos de datos** (*INT*, *VARCHAR*, *DATE*) y restricciones (*CHECK*, *ENUM*).

2.6. Tuplas

Una tupla es **una fila**: un alumno concreto.

- Su **orden** no importa: la fila 1 y la 4 pueden intercambiarse.
- **No hay dos tuplas iguales.**

2.7. Definición formal (para los más curiosos)

Una relación **R** tiene dos partes:

Parte	Qué es	¿Cambia?	Ejemplo
Cabecera (o esquema)	Pares <i>atributo : dominio</i>	No (casi nunca)	(nombre : texto)
Cuerpo (o extensión)	Conjunto de tuplas	Sí, constantemente	{(nombre: Aya), ...}

```

ALUMNOS { (id_alumno:entero), (nombre:texto), (apellidos:texto),
          (fecha_nac:fecha), (localidad:texto) }           ← cabecera

{ (id_alumno:1), (nombre:Aya), (apellidos:Essadik),
  (fecha_nac:2007-03-12), (localidad:Ubrique) }          ← una tupla

```

Como es un **conjunto** de pares, el orden no importa: estas dos tuplas son **la misma**:

```

{ (nombre:Aya), (id_alumno:1), (localidad:Ubrique), ... }
{ (id_alumno:1), (localidad:Ubrique), (nombre:Aya), ... }

```

2.8. Las 6 propiedades de toda relación

1. 🏷️ Tiene un **nombre único**.
2. 🧩 Los valores son **atómicos** (una celda = un valor). Se dice que está **normalizada**.
3. 📄 No hay dos **atributos** con el mismo nombre.
4. ↔ El orden de los **atributos** no importa.
5. 🚫 No hay **tuplas duplicadas**.
6. ⬆ El orden de las **tuplas** no importa.

¿Qué es un valor "no atómico"?

✗ **Incorrecto (grupo repetitivo)**

✅ **Correcto**

telefonos = "956111111, 600222222" Una fila por teléfono, o una tabla **TELEFONOS** aparte.

modulos = "BD, Prog, LM" Tabla **MATRICULAS** (la veremos en el apartado 6).

2.9. Relación frente a tabla de SQL

Aspecto	Relación (teoría)	Tabla SQL (práctica)
Filas duplicadas	Prohibidas	Posibles si no hay clave primaria
Orden de columnas	Irrelevante	Existe (orden de creación)
Dominios	Completos	Aproximados con tipos y CHECK
NULL	Discutido por Codd	Soportado

🌀 **Conclusión:** una tabla SQL **bien diseñada** se comporta como una relación. Las reglas de este tema son las que la "disciplinan".

2.10. Dos tipos de relaciones

Tipo	Qué es	¿Guarda datos propios?	Analogía
Relación base	Tabla real del sistema.	✓ Sí	El archivador
Vista	Tabla virtual definida por una consulta.	✗ No	Una ventana a través de la que miras el archivador

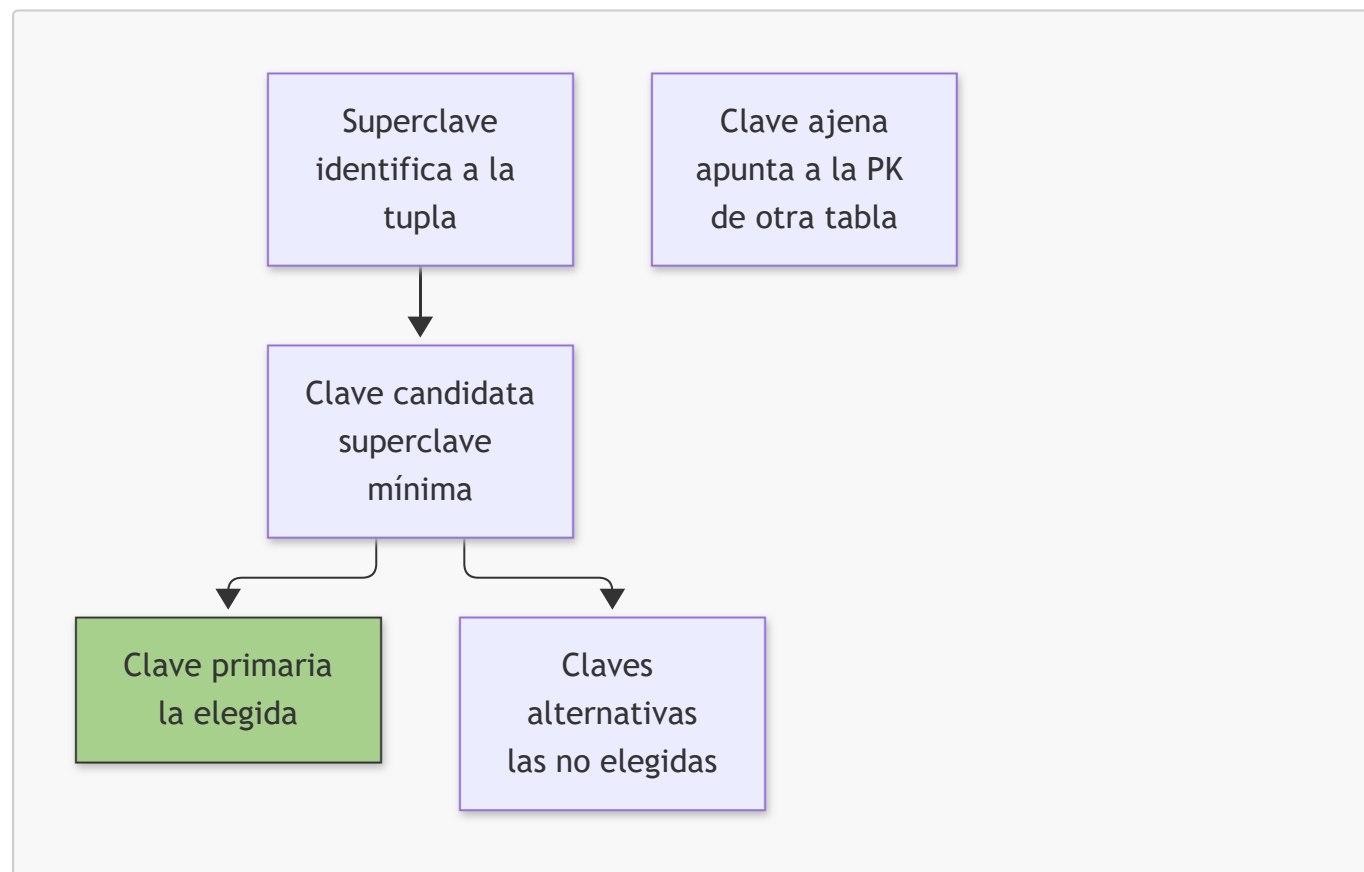
```
-- Vista: solo los alumnos de Ubrique (no copia datos, los "mira")
CREATE VIEW alumnos_ubrique AS
SELECT id_alumno, nombre, apellidos
FROM alumnos
WHERE localidad = 'Ubrique';
```

(Las vistas se estudian a fondo en el Tema 6; RA2.f.)

3. Las claves: el DNI de las filas

3.1. El problema de los gemelos

Si dos alumnos se llaman **María García López**, ¿cómo los distinguimos? Necesitamos algo que **identifique sin dudas** a cada fila: una **clave**.



3.2. Superclave y clave candidata

Concepto	Definición	Ejemplo en ALUMNOS
Superclave	Cualquier conjunto de atributos que identifica de forma única.	{id_alumno}, {dni}, {id_alumno, nombre}, {dni, apellidos}...
Clave candidata	Superclave minimal (no se le puede quitar nada).	{id_alumno} y {dni}

Para ser **clave candidata**, un conjunto K debe cumplir:

1. **Unicidad:** nunca hay dos tuplas con el mismo valor de K.
2. **Irreducibilidad (minimalidad):** si quitas un atributo de K, deja de ser única.

✳ **Ejemplo:** {id_alumno, nombre} es superclave pero **no** candidata: nombre sobra, porque id_alumno ya basta.

3.3. Cómo reconocer una clave candidata

⚠ **Error típico:** mirar los datos de hoy y decir "no hay nombres repetidos, luego nombre es clave".

La clave se decide por el **significado real** del atributo, no por los datos actuales.

Atributo	Datos de hoy	¿Clave candidata?	Razón
nombre	Todos distintos	✗	Mañana puede entrar otra "Aya".
localidad	Repetidos	✗	Es evidente: varios viven en Ubrique.
dni	Distintos	✓	Por ley, es único por persona.
id_alumno	Distintos	✓	Lo asigna el sistema.

🔍 **Regla de oro:** un **duplicado** demuestra que algo **no** es clave. La **ausencia** de duplicados **no** demuestra que lo sea.

3.4. Clave primaria (PK)

La **clave primaria** es la candidata **elegida** para identificar las filas.

Propiedad	¿Se cumple?
Única	✓
No admite NULL	✓
Una sola por tabla	✓
Puede ser compuesta	✓

¿Cómo elegir entre varias candidatas?

1. La que mejor identifique la tupla en el contexto.
2. La más **corta** (menos atributos).

3. Preferiblemente **numérica con autoincremento** (ocupa poco y es rápida).
4. **Estable**: que su valor no cambie nunca.

🎓 **Caso real:** el DNI es candidata. ¿Por qué usamos `id_alumno` como PK?

- Alumnos extranjeros pueden no tener DNI español.
- Un DNI mal escrito se corrige; si fuera PK, habría que cambiarlo **en todas las tablas relacionadas**.
- Un número entero ocupa menos que un `CHAR(9)`.

3.5. Claves alternativas

Las candidatas **no elegidas**. Siguen identificando, pero no son la PK.

Tabla	Candidatas	PK elegida	Alternativas
ALUMNOS	<code>id_alumno, dni</code>	<code>id_alumno</code>	<code>dni</code>
PROFESORES	<code>id_profesor, dni, email</code>	<code>id_profesor</code>	<code>dni, email</code>
MODULOS	<code>id_modulo, codigo, (nombre, curso)</code>	<code>id_modulo</code>	<code>codigo, (nombre, curso)</code>

3.6. Claves compuestas

Cuando **un solo atributo no basta**, se combinan varios.

Ejemplo: la tabla `MATRICULAS`. Un alumno se matricula en **muchos módulos**, y un módulo tiene **muchos alumnos**.

<code>id_alumno</code>	<code>id_modulo</code>	<code>nota</code>
1	10	8.5
1	11	7.0
2	10	5.5

- `id_alumno` solo → se repite (el 1 aparece dos veces). ❌
- `id_modulo` solo → se repite. ❌
- `(id_alumno, id_modulo)` → cada pareja es única. ✅

💡 "Un alumno solo puede matricularse **una vez** en el mismo módulo." Esa frase es la regla que convierte la pareja en clave.

3.7. Claves ajenas (FK): los hilos que unen las tablas

Una **clave ajena** es un atributo cuyos valores **deben existir** como clave primaria en otra tabla (o en la misma). Es **el mecanismo que crea las relaciones**.

PROFESORES

id_profesor (PK)	nombre
1	Juan C.
2	Laura
3	Pedro

MODULOS

id_modulo (PK)	nombre	id_profesor (FK)
10	Bases Datos	1
11	Programación	2
12	Lenguajes M.	2

Relación: `MODULOS.id_profesor` (FK) → `PROFESORES.id_profesor` (PK). Un profesor puede impartir varios módulos; Pedro no imparte ninguno, lo cual es válido.

📌 **Analogía:** la clave ajena es como una **dirección postal**. Solo tiene sentido si esa dirección **existe en el callejero**.

Cómo "seguir el hilo" (lo hace SQL con `JOIN`):

1. Miras el módulo 10 → `id_profesor = 1`.
2. Vas a `PROFESORES` y buscas el 1.
3. ¡Es Juan C.! Ya sabes quién imparte Bases de Datos.

PK frente a FK: ¿se pueden repetir?

	Clave primaria	Clave ajena
¿Valores repetidos?	✗ Nunca	✓ Sí (varios módulos por profesor)
¿NULL?	✗ Nunca	✓ Si se permite
¿Mismo nombre que la PK?	—	No obligatorio, pero muy recomendable

Relación reflexiva: una tabla que apunta a sí misma

En `PROFESORES`, cada profesor puede tener un **jefe**, que **también es profesor**.

id_profesor	nombre	id_jefe
1	Juan C.	NULL
2	Laura	1
3	Pedro	1

`id_jefe` es FK hacia `PROFESORES.id_profesor`. Juan C. no tiene jefe → `NULL`.

3.8. Cuadro comparativo de claves

Clave	Cuántas por tabla	¿NULL?	¿Duplicados?	SQL
Primaria	Una	No	No	PRIMARY KEY
Alternativa	Varias	Sí*	No	UNIQUE
Ajena	Varias	Sí, si se permite	Sí	FOREIGN KEY ... REFERENCES

* Aunque **UNIQUE** admite **NULL**, en el diseño real suele combinarse con **NOT NULL**.

4. El esquema: el plano de la base de datos

4.1. Esquema y estado

 **Analogía:** el **esquema** es el **plano** de un edificio; el **estado** es **quién vive en cada piso hoy**.

	Esquema	Estado
Qué es	Estructura (tablas, columnas, claves)	Datos en un instante
¿Cambia?	Rara vez	Constantemente
Equivale a	Cabecera de las relaciones	Cuerpo de las relaciones

4.2. Qué debe contener un esquema

- 🏠 Nombre de las **relaciones**. (MAYÚSCULAS)
- 📄 Sus **atributos**. (minúsculas)
- 🌐 Los **dominios** de cada atributo. (no se representa)
- 🔑 Las **claves primarias**. (# + negrita + subrayado)
- 🔑 Las **claves alternativas**. (subrayado simple o doble)
- 🔗 Las **claves ajenas**. (subrayado punteado + descripción)
- 🔗 Los **atributos opcionales** (se les adjunta un '*')

4.3. Descripción de las tablas

Tabla	PK	Claves ajenas	Alternativas
DEPARTAMENTOS	id_depto	—	nombre
PROFESORES	id_profesor	id_depto → DEPARTAMENTOS; id_jefe → PROFESORES	dni
MODULOS	id_modulo	id_profesor → PROFESORES	—
ALUMNOS	id_alumno	—	dni
MATRICULAS	(id_alumno , id_modulo)	id_alumno → ALUMNOS; id_modulo → MODULOS	—

 **Observa MATRICULAS:** sus dos claves ajenas **forman** la clave primaria. Es la típica tabla que resuelve una relación **muchos a muchos**.

4.4. Nuestro esquema

DEPARTAMENTOS (**#id_depto**, nombre)

PROFESORES (**#id_profesor**, nombre, dni, id_depto, id_jefe)

- PROFESORES.id_depto → DEPARTAMENTOS.id_depto
- PROFESORES.id_jefe → PROFESORES.id_profesor

MODULOS (**#id_modulo**, nombre, horas, id_profesor)

- MODULOS.id_profesor → PROFESORES.id_profesor

ALUMNOS (**#id_alumno**, nombre, apellidos, dni, localidad)

MATRICULAS(**#id_alumno**, **#id_modulo**, nota)

- MATRICULAS.id_alumno → ALUMNOS.id_alumno
- MATRICULAS.id_modulo → MODULOS.id_modulo

4.5. Un estado posible de la base de datos

DEPARTAMENTOS

id_depto	nombre
1	Informática
2	Inglés

PROFESORES

id_profesor	nombre	dni	id_depto	id_jefe
1	Juan C.	11111111A	1	NULL
2	Laura	22222222B	1	1
3	Pedro	33333333C	2	NULL

MODULOS

id_modulo	nombre	horas	id_profesor
10	Bases de Datos	105	1
11	Programación	230	2
12	Inglés técnico	60	3

ALUMNOS

id_alumno	nombre	apellidos	dni	localidad
1	Aya	Essadik	44444444D	Ubrique
2	César	Gallardo	55555555E	El Bosque
3	Maitane	Navarro	66666666F	Ubrique

MATRICULAS

id_alumno	id_modulo	nota
1	10	8.5
1	11	7.0
2	10	5.5
3	12	NULL

 **Pregunta:** ¿qué significa que la nota de Maitane sea **NULL**? → *Todavía no ha sido evaluada.* No es un cero.

5. Reglas de integridad: que los datos no mientan

5.1. Por qué importan

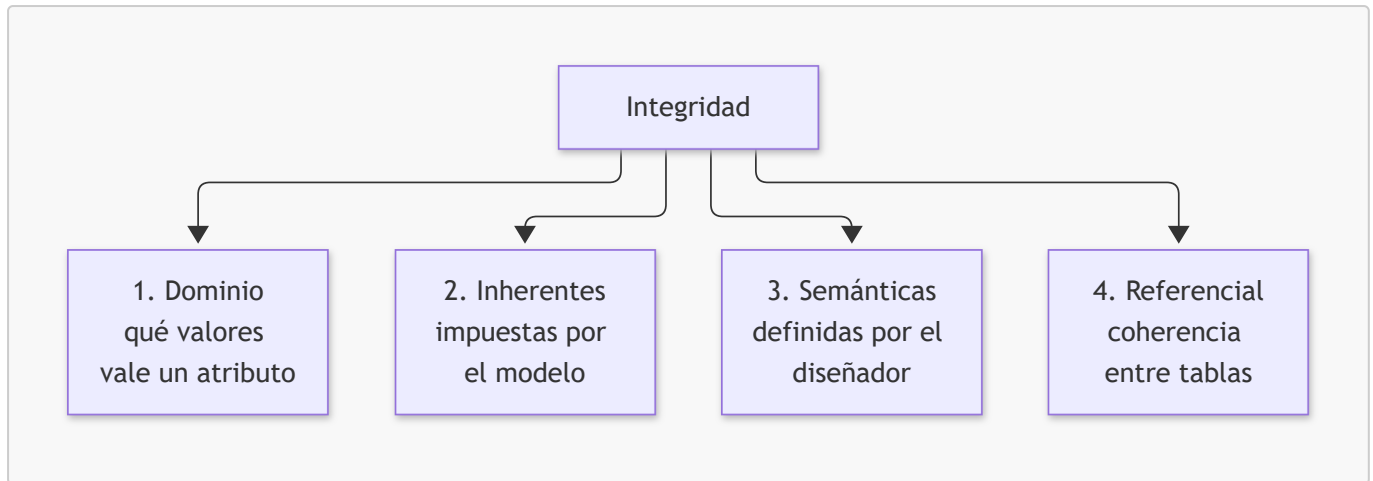
Una base de datos **sin reglas** es un cuaderno donde cualquiera escribe lo que quiere.

Ejemplos de datos **absurdos** que las reglas evitan:

-  Un alumno con nota **47**.
-  Una matrícula de un alumno que **no existe**.
-  Dos alumnos con el **mismo DNI**.
-  Un módulo impartido por un profesor **inexistente**.

 Las reglas de integridad son la **seguridad del sistema**: se cumplen **siempre**, en **todos** los estados.

5.2. Los cuatro niveles



Nivel	Quién la define	Ejemplo
Dominio	Diseñador	nota es decimal.
Inherente	El propio modelo	No hay filas duplicadas.
Semántica	Diseñador	La nota está entre 0 y 10.
Referencial	Diseñador	La matrícula referencia un alumno real.

5.3. El valor nulo (NULL)

NULL significa "**no hay información**". Puede ser porque:

- 🤖 **Se desconoce** (todavía no hay nota).
- 🚫 **No aplica** (un alumno sin segundo apellido).

Valor	¿Qué significa?	¿Es NULL?
0	La nota es cero	✗
' ' (vacío)	Texto vacío	✗
NULL	No se sabe / no aplica	✓

🍕 **Analogía:** pregunto «¿cuántas pizzas has comido hoy?».

- 0 → "ninguna" (lo sé con certeza).
- NULL → "no lo sé / no te lo he preguntado".

Cuidado en SQL: NULL no es un valor, así que **no se compara con =**.

```

SELECT * FROM matriculas WHERE nota = NULL;    -- ✗ no devuelve nada nunca
SELECT * FROM matriculas WHERE nota IS NULL;   -- ✓ alumnos sin evaluar
SELECT * FROM matriculas WHERE nota IS NOT NULL; -- ✓ alumnos con nota
  
```

? **Curiosidad:** NULL = NULL **no es verdadero**: es "desconocido". ¿Cómo vas a saber si dos desconocidos son iguales?

5.4. Reglas inherentes al modelo

Las impone el modelo relacional; **no hay que escribirlas**:

-  **Integridad de entidad:** ningún atributo de la PK puede ser **NULL**.
-  **Cada relación (tabla) tiene un nombre único.**
-  **Cada atributo (columna) tiene un nombre único dentro de su relación.**
-  **El esquema es fijo:** todas las filas de una relación tienen los mismos atributos.
-  **Cada atributo tiene un dominio:** sus valores pertenecen al conjunto de valores permitido para esa columna.
-  **Un valor por celda:** cada atributo de una tupla contiene un único valor de su dominio, no una lista ni un grupo de valores.
-  **No hay tuplas (filas) duplicadas:** una relación es un conjunto de tuplas.
-  **El orden no importa:** ni el de las tuplas ni el de los atributos.
-  **Las claves candidatas son irreducibles:** no se puede quitar ningún atributo y seguir identificando de forma única las tuplas.

? ¿Por qué la PK no puede ser **NULL**?

La PK sirve para **identificar**. Una fila cuya identificación es "desconocida" es una fila que nadie puede localizar. Si en una clave compuesta una parte fuese **NULL**, esa parte no sería necesaria para distinguir, y la clave dejaría de ser mínima.

5.5. Restricciones semánticas (las que define el diseñador)

Mapa de las restricciones

Restricción	Pregunta que responde	SQL
Clave primaria	¿Cómo identifico cada fila?	PRIMARY KEY
Clave alternativa	¿Qué otro dato no puede repetirse?	UNIQUE
Obligatoriedad	¿Es obligatorio rellenarlo?	NOT NULL
Verificación	¿Cumple una condición?	CHECK
Integridad referencial	¿Apunta a algo que existe?	FOREIGN KEY
Aserción	¿Se cumple una condición entre varias tablas?	CREATE ASSERTION
Disparador	¿Hay que reaccionar automáticamente?	CREATE TRIGGER

Las cinco primeras son **declarativas** ("esto debe cumplirse"). El disparador es **procedimental** ("haz esto cuando ocurra aquello").

a) **PRIMARY KEY**: el carné de identidad

```
id_alumno INT PRIMARY KEY
```

-  Sin duplicados ·  Sin **NULL** ·  Solo una por tabla.

b) UNIQUE: el segundo identificador

```
dni CHAR(9) UNIQUE
```

- ✗ Sin duplicados · ✓ Admite NULL · ✓ Puede haber varias.

id_alumno	dni	Resultado
1	44444444D	✓
2	55555555E	✓
3	44444444D	✗ DNI duplicado

c) NOT NULL: "este dato es obligatorio"

```
nombre VARCHAR(30) NOT NULL
```

Columna	¿Obligatoria?	Motivo
nombre	✓	Un alumno sin nombre no tiene sentido.
dni	✓	Es un dato legal.
segundo_apellido	✗	Hay personas que no lo tienen.

d) CHECK: el filtro de calidad

```
nota DECIMAL(4,2) CHECK (nota BETWEEN 0 AND 10)
```

Intento	Resultado
nota = 7.5	✓
nota = 47	✗ Rechazado
nota = -3	✗ Rechazado

⚙️ **Nota técnica:** MySQL aplica CHECK desde la versión **8.0.16**. En anteriores lo ignoraba.

e) FOREIGN KEY: se desarrolla en el apartado 6.**f) Aserciones y disparadores**

	Aserción	Disparador
Qué hace	CHECK que afecta a varias tablas	Ejecuta código ante INSERT , UPDATE o DELETE
Tipo	Declarativa	Procedimental
¿En MySQL?	✗ No existe	✓ Sí (CREATE TRIGGER)
Se estudia en...	—	Tema 8 (RA5.h)

Ejemplo de aserción (regla de varias tablas): *"Un profesor no puede impartir más de 500 horas en total."* Como MySQL no tiene aserciones, se implementaría con un **disparador**.

g) Restricciones que no caben en el diseño (RA6.h)

Algunas reglas **no pueden expresarse** en el esquema relacional y hay que **documentarlas** aparte:

Regla de negocio	¿Dónde se documenta / implementa?
"Un alumno no puede matricularse en más de 8 módulos"	Documentación + disparador o aplicación
"La nota solo se puede poner tras la fecha de examen"	Disparador / lógica de la aplicación
"Un jefe de departamento debe ser del mismo departamento"	Disparador

El instituto completo con restricciones

```
CREATE TABLE alumnos (
  id_alumno INT PRIMARY KEY,
  nombre VARCHAR(30) NOT NULL,
  apellidos VARCHAR(60) NOT NULL,
  dni CHAR(9) NOT NULL UNIQUE,
  localidad VARCHAR(30)
);

CREATE TABLE matriculas (
  id_alumno INT,
  id_modulo INT,
  nota DECIMAL(4,2) CHECK (nota BETWEEN 0 AND 10),
  PRIMARY KEY (id_alumno, id_modulo),
  FOREIGN KEY (id_alumno) REFERENCES alumnos(id_alumno),
  FOREIGN KEY (id_modulo) REFERENCES modulos(id_modulo)
);
```

6. Integridad referencial: nadie puede apuntar a un fantasma

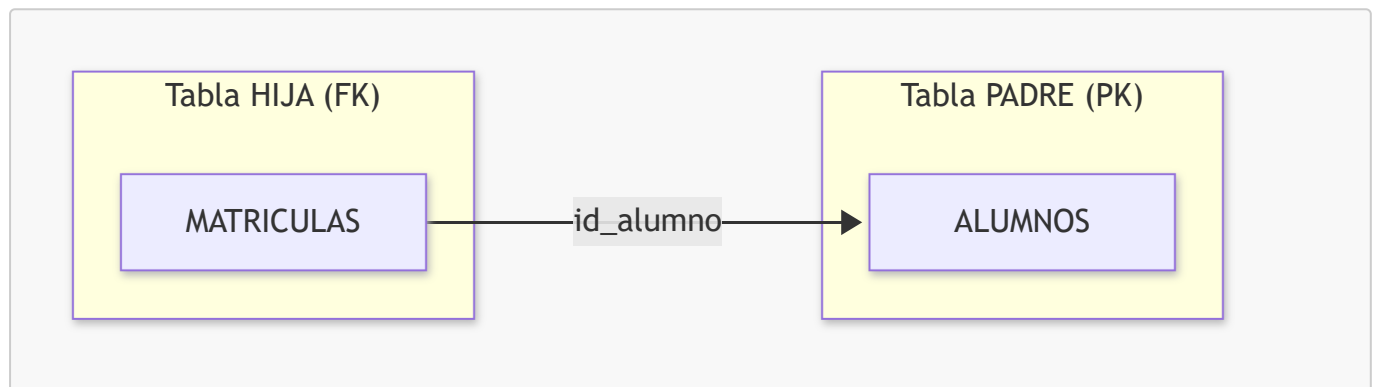
6.1. La regla

 **Regla de integridad referencial:** cada valor de una clave ajena debe existir como clave primaria en la tabla referenciada, o bien ser **NULL**.

Ejemplo con el instituto:

id_alumno	id_modulo	nota	¿Válida?
1	10	8.5	✓ Alumno 1 y módulo 10 existen.
2	10	5.5	✓
9	10	6.0	✗ No existe el alumno 9 (matrícula "fantasma").

6.2. ¿Cuándo se puede romper la regla?



Operación	¿Puede violar la regla?	Por qué
INSERT en la hija	✓ Sí	La FK podría no existir en el padre.
UPDATE de la FK en la hija	✓ Sí	El nuevo valor podría no existir.
DELETE en el padre	✓ Sí	Los hijos quedarían huérfanos .
UPDATE de la PK en el padre	✓ Sí	Los hijos quedarían huérfanos .
INSERT en el padre	✗ No	—
DELETE en la hija	✗ No	—

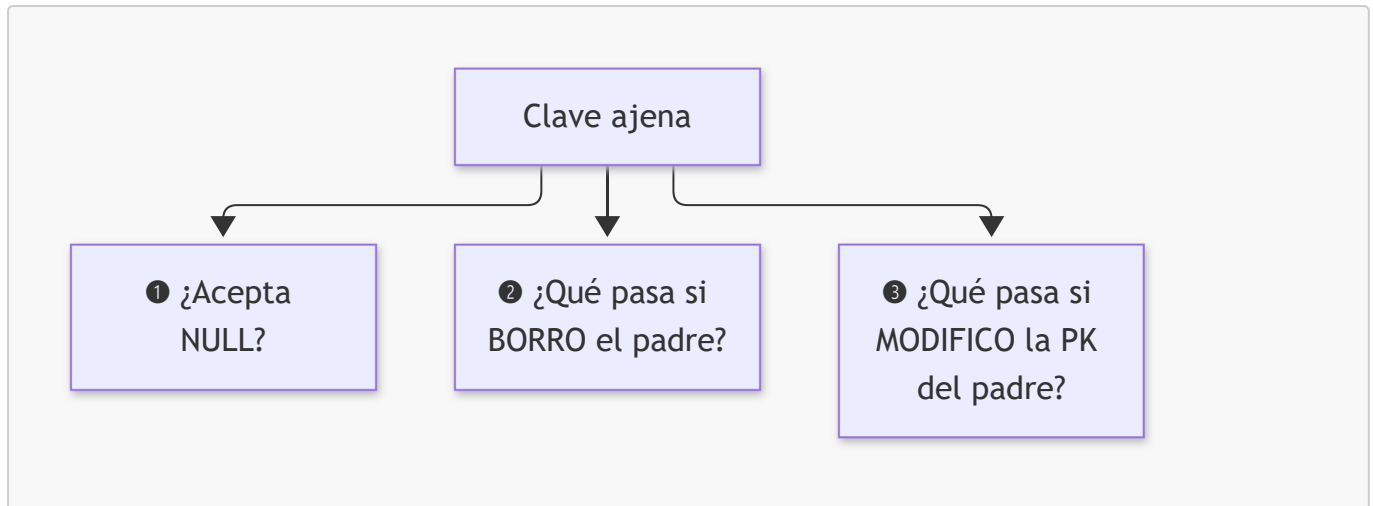
 **Huérfano:** fila hija cuya FK apunta a un padre que ya no existe.

Ante una operación peligrosa hay **dos salidas**:

1.  **Rechazarla.**
2.  **Aceptarla y compensar** (arreglar lo que haga falta).

6.3. Las tres preguntas de cada clave ajena

Para **cada** FK, el diseñador debe contestar:



1 Regla de los nulos

Respuesta	Significado	Ejemplo
Acepta NULL	La relación es opcional.	Un profesor puede no tener jefe (<code>id_jefe</code>).
NOT NULL	La relación es obligatoria.	Una matrícula siempre tiene alumno.

2 y 3 Reglas de borrado y modificación

Opción	Siglas	¿Qué hace?	SQL
Restringir	B:R · M:R	Prohíbe la operación.	<code>RESTRICT / NO ACTION</code>
Propagar	B:C · M:C	Arrastra el cambio a los hijos.	<code>CASCADE</code>
Anular	B:N · M:N	Pone la FK de los hijos a NULL .	<code>SET NULL</code>
Valor por defecto	B:D · M:D	Pone la FK de los hijos a su valor por defecto.	<code>SET DEFAULT</code>

⚠ En **MySQL/InnoDB** `SET DEFAULT` se reconoce en la sintaxis pero **no se aplica**. Las opciones reales son `RESTRICT`, `NO ACTION`, `CASCADE` y `SET NULL`.

6.4. Veámoslo en acción: "se da de baja un alumno"

Datos de partida:

ALUMNOS

(1, Aya) · (2, César)

MATRICULAS (`id_alumno`, `id_modulo`, `nota`)

(1, 10, 8.5) · (1, 11, 7.0) · (2, 10, 5.5)

Ejecutamos: `DELETE FROM alumnos WHERE id_alumno = 1;`

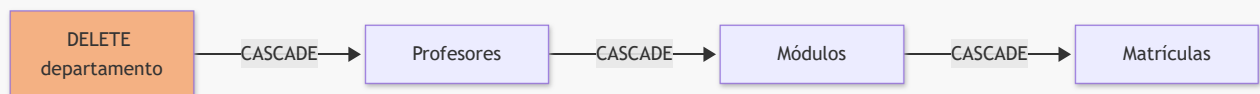
Política de borrado	¿Qué ocurre?	Matrículas que quedan
RESTRICT	❌ Error. Aya tiene matrículas; no se puede borrar.	(1,10) (1,11) (2,10)
CASCADE	✅ Se borra Aya y sus matrículas.	(2,10)
SET NULL	✅ Se borra Aya; sus matrículas quedan con <code>id_alumno = NULL</code> .	(NULL,10) (NULL,11) (2,10)

🤔 **Reflexión:** `SET NULL` aquí sería un **desastre**: matrículas sin alumno no sirven de nada (y además la FK forma parte de la PK, que **no puede ser NULL**). Por eso, para `MATRICULAS.id_alumno` se elige **CASCADE** (si se va el alumno, su matrícula pierde sentido) o **RESTRICT** (para obligar a revisar antes).

6.5. ¿Qué política elijo? Guía rápida

Situación	Política recomendada	Razonamiento
Datos importantes (facturas, notas)	RESTRICT	Evita borrados accidentales.
Datos dependientes que no tienen sentido solos (líneas de factura, matrículas)	CASCADE	Si desaparece el padre, el hijo sobra.
Relación opcional (jefe de un profesor)	SET NULL	El hijo sigue existiendo, solo pierde la relación.
La PK del padre cambia (raro)	CASCADE en modificación	Mantiene todo sincronizado.

🔪 **Peligro de CASCADE en borrado:** borrar un departamento podría arrastrar **profesores** → **módulos** → **matrículas**. Una sola orden, ¡y desaparece medio instituto! Úsalo con cabeza.



6.6. En SQL

```

CREATE TABLE modulos (
  id_modulo  INT PRIMARY KEY,
  nombre     VARCHAR(50) NOT NULL,
  horas      INT,
  id_profesor INT,
  asignar    -- acepta NULL: aún sin
  FOREIGN KEY (id_profesor) REFERENCES profesores(id_profesor)
  ON DELETE SET NULL -- si se va el profesor, el
                    módulo queda "sin profesor"
  ON UPDATE CASCADE -- si cambia su id, se
  
```

```
actualiza aquí
);
```

7. El grafo relacional: el plano del arquitecto

7.1. ¿Qué es?

Es la forma **textual** de escribir el esquema lógico, con **claves** y **políticas de integridad**. Es el resultado de pasar el diagrama E/R al modelo relacional (Tema 4, RA6).

7.2. Notación

Elemento	Se escribe	Ejemplo
Clave primaria	# delante (negrita y subrayado en papel)	#id_alumno
Clave alternativa	Doble subrayado	dni
Atributo opcional	Con asterisco *	email*
Clave ajena	TABLA.atributo -> TABLA_REFERENCIADA	MODULOS.id_profesor -> PROFESORES
Políticas	(nulos; B:x, M:x)	(NULL; B:N, M:C)

Códigos:

Borrado	Modificación	Significado
B:R	M:R	Restringido
B:C	M:C	En cascada
B:N	M:N	A nulo
B:D	M:D	A valor por defecto

7.3. Cómo leer una línea

```
MODULOS.id_profesor -> PROFESORES (NULL; B:N, M:C)
```

Fragmento	Se lee
MODULOS.id_profesor	La FK id_profesor de la tabla MODULOS...
-> PROFESORES	...apunta a la PK de PROFESORES.
NULL	Admite nulos.
B:N	Si se borra el profesor, el módulo queda sin profesor.

Fragmento	Se lee
M:C	Si cambia el id del profesor, se actualiza en el módulo.

7.4. Grafo del instituto completo

DEPARTAMENTOS (#id_depto, nombre)

PROFESORES (#id_profesor, nombre, dni, id_depto, id_jefe*)

- PROFESORES.id_depto → DEPARTAMENTOS.id_depto (NOT NULL; B:R, M:C)
- PROFESORES.id_jefe → PROFESORES.id_profesor (NULL; B:N, M:C)

MODULOS (#id_modulo, nombre, horas, id_profesor*)

- MODULOS.id_profesor → PROFESORES.id_profesor (NULL; B:N, M:C)

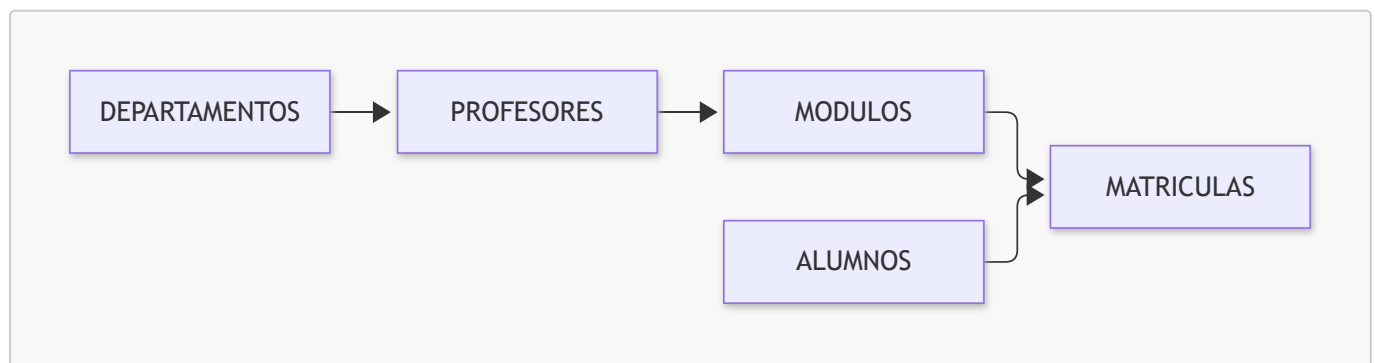
ALUMNOS (#id_alumno, nombre, apellidos, dni, localidad*)

MATRICULAS(#id_alumno, #id_modulo, nota*)

- MATRICULAS.id_alumno → ALUMNOS.id_alumno (NOT NULL; B:C, M:C)
- MATRICULAS.id_modulo → MODULOS.id_modulo (NOT NULL; B:R, M:C)

7.5. Del grafo al SQL (paso a paso)

Paso	Qué haces
1	Cada relación → un CREATE TABLE .
2	Cada atributo → una columna con su tipo.
3	# → PRIMARY KEY .
4	* → sin NOT NULL ; sin asterisco → NOT NULL .
5	Cada flecha -> → un FOREIGN KEY ... REFERENCES .
6	B:x y M:x → ON DELETE y ON UPDATE .
7	Orden de creación: primero los padres, luego los hijos .



 Es como construir un edificio: primero los cimientos (tablas independientes), luego las plantas que se apoyan en ellos.

8. Caso práctico completo

Diseñemos paso a paso una **biblioteca** del instituto.

Enunciado: Los socios piden prestados libros. Cada libro pertenece a una editorial. Un préstamo tiene fecha de salida y devolución.

Paso 1. Detectar las relaciones

Relación	Atributos
EDITORIALES	id_editorial, nombre
LIBROS	id_libro, titulo, isbn, id_editorial
SOCIOS	id_socio, nombre, email
PRESTAMOS	id_socio, id_libro, fecha_salida, fecha_devolucion

Paso 2. Elegir claves

Relación	PK	Alternativas	Razón
EDITORIALES	id_editorial	nombre	Numérica y estable.
LIBROS	id_libro	isbn	El ISBN es único pero largo.
SOCIOS	id_socio	email	Email único.
PRESTAMOS	(id_socio, id_libro, fecha_salida)	—	Un socio puede repetir el préstamo de un libro en otra fecha .

Paso 3. Elegir políticas

FK	¿NULL?	Borrado	Modificación	Justificación
LIBROS.id_editorial	No	R	C	No se borra una editorial con libros.
PRESTAMOS.id_socio	No	R	C	No se borra un socio con historial.
PRESTAMOS.id_libro	No	R	C	Ídem.

Paso 4. Grafo relacional

```
EDITORIALES(#id_editorial, nombre)
LIBROS(#id_libro, titulo, isbn, id_editorial)
    LIBROS.id_editorial -> EDITORIALES (NOT NULL; B:R, M:C)
SOCIOS(#id_socio, nombre, email)
PRESTAMOS(#id_socio, #id_libro, #fecha_salida, fecha_devolucion*)
```

```
PRESTAMOS.id_socio -> SOCIOS (NOT NULL; B:R, M:C)
PRESTAMOS.id_libro -> LIBROS (NOT NULL; B:R, M:C)
```

Paso 5. SQL

```
CREATE TABLE editoriales (
  id_editorial INT PRIMARY KEY,
  nombre VARCHAR(60) NOT NULL UNIQUE
);

CREATE TABLE libros (
  id_libro INT PRIMARY KEY,
  titulo VARCHAR(100) NOT NULL,
  isbn CHAR(13) NOT NULL UNIQUE,
  id_editorial INT NOT NULL,
  FOREIGN KEY (id_editorial) REFERENCES editoriales(id_editorial)
  ON DELETE RESTRICT ON UPDATE CASCADE
);

CREATE TABLE socios (
  id_socio INT PRIMARY KEY,
  nombre VARCHAR(60) NOT NULL,
  email VARCHAR(80) NOT NULL UNIQUE
);

CREATE TABLE prestamos (
  id_socio INT,
  id_libro INT,
  fecha_salida DATE,
  fecha_devolucion DATE,
  PRIMARY KEY (id_socio, id_libro, fecha_salida),
  FOREIGN KEY (id_socio) REFERENCES socios(id_socio)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  FOREIGN KEY (id_libro) REFERENCES libros(id_libro)
  ON DELETE RESTRICT ON UPDATE CASCADE,
  CHECK (fecha_devolucion IS NULL OR fecha_devolucion >= fecha_salida)
);
```

9. Errores típicos (¡no caigas!)

✗ Error

Usar **nombre** como clave primaria.

Guardar varios valores en una celda ("BD, Prog").

Comparar con = **NULL**.

Confundir 0 o '' con **NULL**.

✓ Corrección

Usa un identificador propio (**id**).

Una fila por valor o una tabla aparte.

Usa **IS NULL**.

NULL es **ausencia** de dato.

✗ Error**✓ Corrección**

Crear la tabla hija antes que la padre.

Padres primero.

Usar **CASCADE** en todo "por si acaso".

Elige la política según el caso.

Crear que un **CHECK** valida contra otras tablas.

Para eso, disparadores.

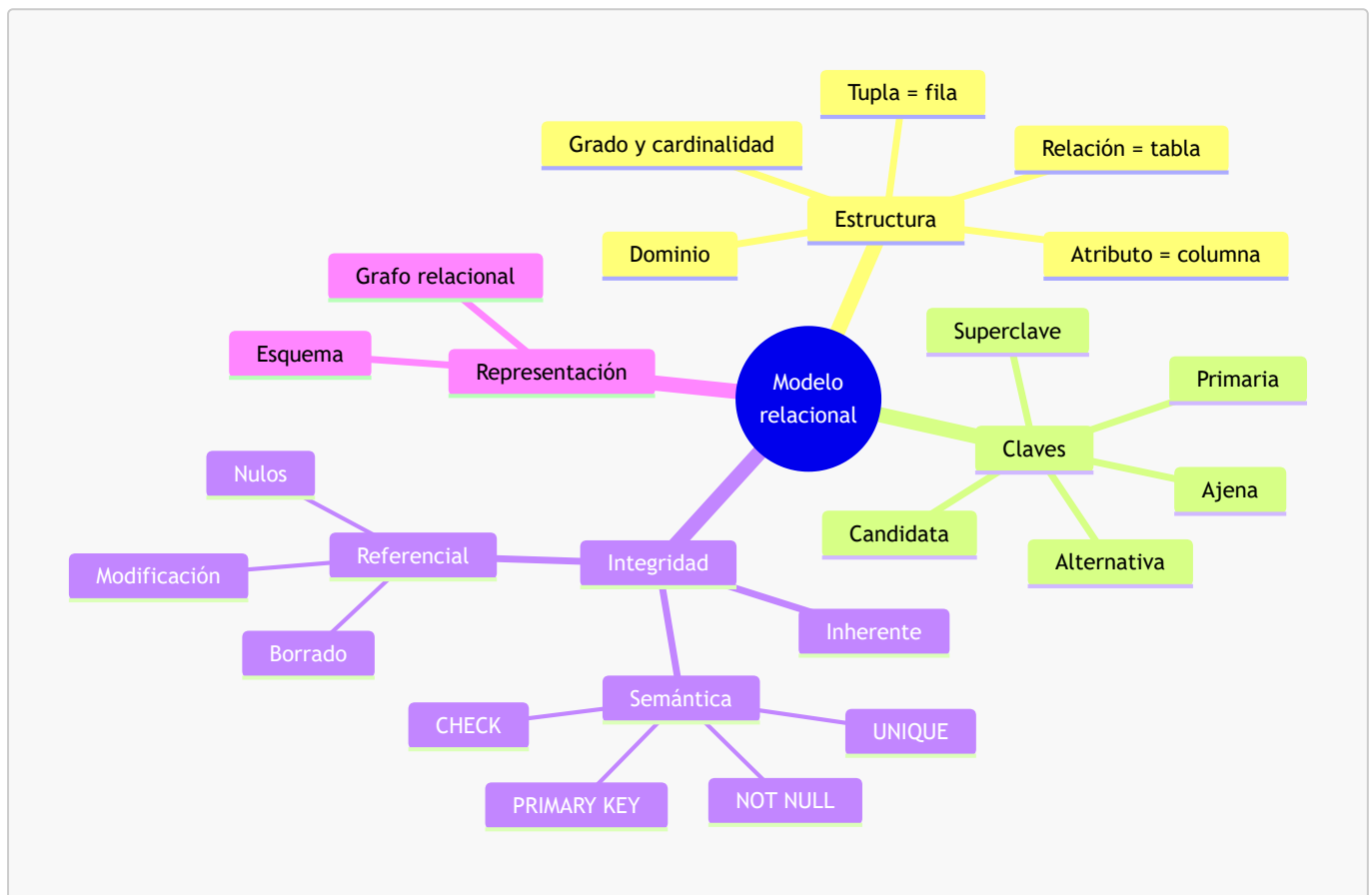
Pensar que FK = PK.

La FK **apunta** a una PK, pero puede repetirse.

Confundir "relación" (Codd) con "relación" (Chen).

Una es **tabla**; la otra, **asociación**.

10. Resumen visual



Ideas clave en una frase

Concepto	Una frase
Relación	Una tabla sin filas repetidas ni orden.
Dominio	Los valores que un atributo puede tomar.
Clave primaria	El DNI de cada fila: único y nunca nulo.
Clave ajena	Un hilo que apunta a la clave primaria de otra tabla.
NULL	Ausencia de dato; no es cero ni vacío.
Integridad referencial	Nadie puede apuntar a algo que no existe.
Grafo relacional	El esquema escrito con sus claves y políticas.

11. Actividades

Nivel 1: comprender

1. Con tus palabras: ¿qué diferencia hay entre una **relación** de Codd y una **relación** del modelo E/R?
2. Indica el **grado** y la **cardinalidad** de la tabla **MODULOS** del apartado 4.5.
3. ¿Por qué un valor "BD, Prog, LM" en una celda viola el modelo relacional?
4. ¿Qué significa que la nota de una matrícula sea **NULL**? ¿Y que sea **0**?

Nivel 2: aplicar

5. En **ALUMNOS(id_alumno, nombre, apellidos, dni, email, localidad)** indica:
 - a) Todas las **claves candidatas** posibles.
 - b) Una **PK** razonada.
 - c) Las **claves alternativas**.
6. Razona si (**nombre, apellidos**) podría ser clave candidata.
7. Dada esta tabla **PEDIDOS(id_pedido, id_cliente, fecha)** y **CLIENTES(id_cliente, nombre)**, escribe el SQL de ambas con la FK, sin permitir pedidos sin cliente.
8. En el apartado 6.4, ¿qué ocurriría al borrar al alumno 2 con **SET NULL** si **id_alumno** forma parte de la PK?

Nivel 3: diseñar

9. Diseña el **grafo relacional** de una **tienda de videojuegos** con **CLIENTES**, **VIDEOJUEGOS**, **PLATAFORMAS** y **VENTAS**. Indica PK, FK, atributos opcionales y las políticas B/M con su justificación.
10. Traduce ese grafo a **CREATE TABLE**.
11. Propón **tres restricciones de negocio** de tu tienda que **no** puedan expresarse en el esquema y di cómo las documentarías o implementarías (RA6.h).

Nivel 4: reflexionar

12. ¿Por qué **CASCADE** en borrado puede ser peligroso? Pon un ejemplo con el instituto.
13. Una tabla SQL sin PK, ¿es una relación en sentido estricto? Razona la respuesta.
14. ¿Qué ventajas tiene usar un **id** numérico como PK en vez del DNI?

12. Autoevaluación rápida (verdadero o falso)

#	Afirmación	V/F
1	Una relación puede tener dos tuplas idénticas.	
2	La PK puede contener valores NULL .	
3	Una tabla puede tener varias claves UNIQUE .	
4	Una FK puede tener valores repetidos.	
5	NULL equivale a cero.	

#	Afirmación	V/F
6	Una vista almacena sus propios datos.	
7	RESTRICT impide borrar un padre con hijos.	
8	El orden de las filas en una relación es relevante.	
9	Toda tabla tiene al menos una clave candidata.	
10	El grado de una relación cambia con cada INSERT .	

► **Ver soluciones**

13. Glosario

Término	Definición
Atómico	Indivisible; una celda guarda un solo valor.
Atributo	Columna de una relación.
Cardinalidad	Número de tuplas.
Clave alternativa	Candidata no elegida como primaria.
Clave ajena (FK)	Atributo(s) que referencian la PK de otra tabla.
Clave candidata	Superclave minimal.
Clave compuesta	Clave formada por varios atributos.
Clave primaria (PK)	Candidata elegida para identificar las tuplas.
Dominio	Conjunto de valores permitidos de un atributo.
Esquema	Estructura de la BD (plano).
Estado	Datos de la BD en un instante.
Grado	Número de atributos.
Grafo relacional	Esquema lógico con claves y políticas de integridad.
Huérfano	Fila hija cuya FK apunta a un padre inexistente.
Integridad referencial	Toda FK existe como PK o es NULL .
NULL	Ausencia de valor.
Relación	Tabla; conjunto de tuplas bajo una cabecera.
Relación reflexiva	Tabla cuya FK apunta a su propia PK.
Superclave	Conjunto de atributos que identifica tuplas.
Tupla	Fila.
Vista	Relación virtual definida por una consulta.

14. Para seguir aprendiendo

-  Apuntes del **Tema 4** (transformación del modelo E/R al relacional) y **Tema 5** (SQL DDL).
-  Artículo original de Codd: *A Relational Model of Data for Large Shared Data Banks* (1970).
-  Prueba en **MySQL Workbench**: *Database* → *Reverse Engineer* para ver el diagrama de una base de datos real (por ejemplo **futbol** o **empresa**).

Normativa: Real Decreto 405/2023, de 29 de mayo (BOE-A-2023-13221) · Real Decreto 686/2010, de 20 de mayo (BOE-A-2010-9269).